

■ 2.6.16 Advanced Topic: Low-Level Output Functions

You can think of `>>` and `>>>` as “high-level” *Mathematica* output functions. They are based on a set of lower-level output primitives. By using the output primitives directly, you can have more control over exactly how *Mathematica* produces output. You will often need to do this, for example, if you write programs that store intermediate data in files.

The basic low-level scheme for writing output in *Mathematica* is as follows. First, you call `OpenWrite` or `OpenAppend` to tell *Mathematica* that you want to write output to a particular file, and to specify in what form that output should be written. Then you call `Write` or `WriteString` to write your sequence of expressions or strings to the file. Finally, you call `Close` to tell *Mathematica* that you are finished with the file.

<code>OpenWrite["file"]</code>	prepare to write output to a file, overwriting the previous contents of the file
<code>OpenAppend["file"]</code>	prepare to append output to a file
<code>OpenWrite["file", options]</code>	open a file, specifying options for how output is to be written to it
<code>Write["file", expr₁, expr₂, ...]</code>	write a sequence of expressions to a file, ending the output with a newline (line feed)
<code>Write[{"file₁", "file₂", ...}, expr₁, ...]</code>	write expressions to a list of files
<code>WriteString["file", str₁, str₂, ...]</code>	write a sequence of character strings to a file, with no extra newlines
<code>Display["file", graphics]</code>	write graphics output to a file, in POSTSCRIPT form
<code>Close["file"]</code>	tell <i>Mathematica</i> that you are finished with a file
<code>ResetMedium["file", options]</code>	reset the options for a file that is already open
<code>ResetMedium[options]</code>	reset options for your standard output

Low-level output functions.

This gets ready to write to the file `test`.
`In[1]:= OpenWrite["test"]`
`Out[1]= test`

This writes a sequence of expressions to `test`, ending the output with a newline (line feed).

```
In[2]:= Write["test", a, b, c]
```

This writes another expression to `test`.

```
In[3]:= Write["test", x]
```

This tells *Mathematica* you have finished writing to the file `test`.

```
In[4]:= Close["test"]
Out[4]= test
```

This is what the file `test` now contains.

```
In[5]:= !!test
abc
x
```

By making a sequence of calls to `Write`, you can output a sequence of expressions to a file. There is some subtlety in the question of exactly how the expressions you give are placed in the file. The main issue is that in standard *Mathematica* output format, expressions have a two-dimensional structure. This means that if you write out a sequence of separate expressions, it is not clear how they should line up. What `Write` does is to output the expressions you give it, ending them with a newline (line feed). The next time you call `Write`, its output goes underneath what it produced before.

`Write` is appropriate for writing out complete *Mathematica* expressions. Sometimes, however, you may want to write out simple character strings, with complete control over where newlines are placed. The function `WriteString` allows you to write out character strings without inserting any extra newlines or other characters. Section 2.6.9 discusses various functions you can use to construct character strings, including the function `ToString`, which converts any *Mathematica* output form to a string.

Open the file `test` for writing.

```
In[5]:= OpenWrite["test"]
Out[5]= test
```

This writes out the expressions `a^2` and `b^2`, with a string of spaces in between.

```
In[6]:= Write["test", a^2, " ", b^2]
```

This closes the file.

```
In[7]:= Close["test"]
Out[7]= test
```

The expressions were all written in input form. As a result, `a^2` and `b^2` appear in a linear format. The string of spaces is displayed with explicit double quotes.

```
In[8]:= !!test
a^2"    "b^2
```

This writes out the same expressions, with the first two now explicitly given in output form.

```
In[8]:= ( OpenWrite["test"];
          Write["test", OutputForm[a^2], OutputForm["  "],
          b^2];
          Close["test"]
)
Out[8]= test
```

The first expression is now given in two-dimensional form, and the spaces have no double quotes around them.

```
In[9]:= !!test
      2
     a  b^2
```

If you are familiar with the C programming language and the UNIX operating system, you will notice that the *Mathematica* functions `OpenWrite`, `OpenAppend`, `Close` and `WriteString` work in much the same way as the UNIX system calls `open`, `close` and `write`. One important difference, however, is that in *Mathematica* you always refer to files by their textual names, rather than through “file descriptors” or “file pointers”.

Another point is that in *Mathematica* all the low-level output primitives can take lists of files, as well as single files. This allows you to write the same output to several files at once. It is often convenient to define an “output channel” that consists of a list of files to which you want to send a particular kind of output. There are several global variables that represent common output channels in *Mathematica*. Functions like `Print` and `Message` use these output channels to determine where to write their output.

<code>\$Output</code>	standard output (used by <code>Print</code>)
<code>\$Echo</code>	files to which to echo each input line
<code>\$Urgent</code>	files for urgent output (syntax error messages, etc.)
<code>\$Messages</code>	files for messages (used by <code>Message</code>)
<code>\$Display</code>	files for graphics output (used by <code>\$DisplayFunction</code>)

Output channels.

If you want to send a particular kind of output to a certain file, you can just add the file to the list for the corresponding output channel. (You can conveniently do this using the function `PrependTo`.) Thus, for example, if you want to keep a record of your complete *Mathematica* session in a file, you can simply add this file to the lists for the output channels `$Echo` and `$Output`. If you want to keep only the input you type, you can just add the file to the list for `$Echo`.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Mathematica supports a few “special files”. On most computer systems, the *Mathematica* files `"stdout"` and `"stderr"` correspond to the standard output and error media, respectively.

Mathematica maintains a list of all the output (and input) files that are open at a particular time. The list is called `$$Media`. You can add and remove entries from this list using `OpenWrite`, `OpenAppend` and `Close`. You cannot, however, modify the list directly.

When you open a file, there are a number of options that you can give to specify how you want output to that file to be formatted. Even once a file is open, you can change the options for it using `ResetMedium`. If you want to know what options are set for a particular file, you can pick out the entry for the file in `$$Media`, and use the function `Options` on it.

You can change the options for your standard output medium by calling `ResetMedium` without giving an explicit file name.

<i>option</i>	<i>default</i>	
<code>FormatType</code>	<code>InputForm</code>	the default output format to use
<code>PageWidth</code>	78	the width of the page
<code>PageHeight</code>	22	the height of the page
<code>TextRendering</code>	<code>Plain</code>	the default outputs text as plain characters; <code>PostScript</code> gives <code>POSTSCRIPT</code>

Some options for output to files.

This opens the file `test`, specifying that the default output format for all output written to it should be `FortranForm`.

```
In[9]:= OpenWrite["test.m", FormatType -> FortranForm]
Out[9]= test.m
```

This writes output to `test`, using the default format type `FortranForm`.

```
In[10]:= Write["test.m", (a + b)^2]
```

This resets the options for `test`, so that future output to it will be done with default format type `InputForm`.

```
In[11]:= ResetMedium["test.m", FormatType -> InputForm]
```

This tells *Mathematica* you have finished using the file `test`.

```
In[12]:= Close["test.m"]
Out[12]= test.m
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This tells *Mathematica* to write textual output to `test` in POSTSCRIPT form.

```
In[13]:= OpenWrite["test.m", TextRendering -> PostScript]
Out[13]= test.m
```

This writes an expression to `test`.

```
In[14]:= Write["test.m", (a + b)^2]
```

The expression was written in a form suitable for input to POSTSCRIPT.

```
In[15]:= !!test.m
0 0 moveto
/Plain findfont setfont
((a + b)^2) show
```

You can use the function `Write` to output any standard *Mathematica* expression. If you want to output graphics, however, you must use `Display` instead.

The option `PageWidth` gives the width of the page available for textual output from *Mathematica*. All lines of output are broken so that they fit in this width. If you do not want any lines to be broken, you can set `PageWidth -> Infinity`. Usually, however, you will want to set `PageWidth` to the value appropriate for your particular output device. On many systems, you will have to run an external program to find out what this value is. Using `SetOptions`, you can make the default rule for `PageWidth` be, for example, `PageWidth :> <<"!devicewidth"`, so that an external program is run automatically to find the value of the option.